A3: LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Interim progress report: work done so far and the detailed plan for what comes next

14 October 2026

Contents

1. Purpose of this document **2**

1.1 How the project is graded (and what that means for the design) 2

2. Starting point: what Stage 1 already had **2**

2.1 Gaps found in Stage 1 3

3. Work completed so far **4**

3.1 Harness (Member A's area) 4

3.2 Evaluation framework (Member C's area) 5

4. Detailed plan for the remaining work **7**

4.1 Stage 2: grammar-aware generator (Member B's area) 7

4.2 Stage 3: LLM-guided loop 8

4.3 Crash triage, minimization and reporting 9

4.4 Configurations, tests, validation 9

4.5 The three PDF guides you asked for 9

5. Honest status summary **9**

1. Purpose of this document

This report covers the work done on the repository today, before Stage 2 is finished. It has three parts:

1. **What the repository contained** when the work started (the Stage 1 baseline).
2. **What has been changed or added so far**, file by file, and *why* each change matters for finding real PostgreSQL bugs and for the grading criteria.
3. **The detailed plan** for the remaining work (Stage 2 grammar fuzzer, Stage 3 LLM loop, crash triage and minimization, bug-report workflow, the three PDF guides).

Nothing in this report is a measured experimental result yet. No fuzzing campaign has been run on the new code, and no bug has been found yet. Every number in the final paper must come from a real run (see `paper/STYLE.md`).

1.1 How the project is graded (and what that means for the design)

Component	Weight	What it needs
Implementation & results	50 %	Full marks only with a **real, previously unknown bug accepted upstream** (merged fix, acknowledged report, CVE).
Progress meetings & final presentation	25 %	Every member can explain every part.
Report (FSE / ACM format)	25 %	Clear motivation, technique, evaluation, related work, threats.

The design decisions below follow from the first row: the tool must be good at *finding new bugs* in PostgreSQL, not only at producing coverage numbers. In practice, that means:

- fuzz an **assert-enabled build of the newest code** (`master` and the newest stable branch), because new code has the most undiscovered bugs, and assertions turn silent corruption into a visible crash;
- catch **wrong results** as well as crashes, because many PostgreSQL optimizer bugs return the wrong rows without crashing;
- make every failure **reproducible, minimal and self-contained**, because PostgreSQL maintainers act quickly on a short SQL script that crashes `master`.

2. Starting point: what Stage 1 already had

The repository has two parts, `src/` (the tool) and `paper/` (the FSE-format report template). Stage 1 had been split into three roles:

Stage 1 task	Owner	State found
Instrumented PostgreSQL + Docker harness, coverage	Member A	`harness/docker/Dockerfile` builds PostgreSQL 18.6 from a release tarball with `--enable-coverage`. `harness/executor.py` runs one query through `psql`.
Query representation, random/mutation baseline	Member B	Only a stand-in `RandomGenerator` in `src/pgfuzz/reference/generators.py`.
Evaluation skeleton, log format, experiment runner	Member C	Complete and well tested: plug-in interfaces, failure oracle, deduplication signatures, SQL feature catalog (148 features), metrics, run-log format v1, experiment runner (paired seeds, resume, parallel), statistics (Mann-Whitney U, Vargha-Delaney A12), report generator, 58 unit tests.

2.1 Gaps found in Stage 1

These gaps would have made the tool miss bugs, or made the evaluation unfair:

1. **No assertions in the server build.** The Dockerfile used `--enable-coverage` but not `--enable-cassert`. Most PostgreSQL bugs found by fuzzers (for example SQLsmith's) are `Assert()` failures. Without `--enable-cassert` they are invisible, or show up later as a random crash with no clue to the cause.
2. **Old code.** A fixed release tarball (18.6) instead of the development branch. Bugs in a released minor version are more likely to be known already.
3. **No server log for the oracle.** The oracle reads `TRAP:`, `PANIC:` and `terminated by signal` from the server log, but the container did not write a log to a place the fuzzer could read.
4. **Coverage was never reset between trials.** gcov counters accumulate in the build tree. Trial 2 would start with trial 1's coverage, which breaks the comparison between arms.
5. **No coverage command.** The executor expects a command that prints coverage as JSON; none existed.
6. **Coverage measurement time was charged to the fuzzing budget.** One `lcov` capture of the backend takes tens of seconds. With one sample per minute, a large part of each 1-hour trial would be spent measuring instead of fuzzing.
7. **Hard-coded paths** (`/home/gow/...`) in `harness/executor.py` and `scripts/`, so they only worked on one laptop.
8. **Only crash-type oracles.** Wrong-result (logic) bugs were not detected at all.
9. **Client-side decoding errors counted as server errors.** psycopg converts values to Python objects. Valid results such as `'infinity'::date` or years after 9999 raise a client-side `DataError`. The executor then reported a *successful* statement as an error, which lowers the measured valid-query rate.

10. **A result-alignment bug in the executor.** If a statement succeeded but the following `RELEASE SAVEPOINT` failed (for example because the server crashed in between), a second result was appended for the same statement, so every later statement result was shifted by one.

3. Work completed so far

3.1 Harness (Member A's area)

`harness/docker/Dockerfile` (rewritten)

Builds PostgreSQL **from git at any branch or tag** and takes build arguments:

Argument	Default	Effect
`PG_REF`	`master`	Branch or tag, e.g. `REL_18_STABLE`, `REL_18_1`.
`CASSERT`	`1`	`--enable-cassert`: `Assert()` failures abort the backend and print `TRAP:` plus a C backtrace.
`COVERAGE`	`0`	`--enable-coverage` (gcov) for the coverage experiments; about 2x slower.
`SANITIZER`	`none`	`asan` (AddressSanitizer) or `ubsan` (UndefinedBehaviorSanitizer).

It always builds with `--enable-debug` and frame pointers (readable backtraces), with libxml, ICU, LZ4, Zstd and OpenSSL (more SQL features to reach), builds contrib modules (`make world-bin`), installs `gdb` and `lcov`, and records the exact source commit (`/opt/SOURCE_COMMIT`) and configure flags (`/opt/CONFIGURE_FLAGS`). Upstream bug reports need both.

`harness/docker/postgresql.fuzz.conf` (new)

Server settings for fuzzing: durability off for speed (`fsync`, `synchronous_commit`, `full_page_writes`), `restart_after_crash = on`, autovacuum off (so plans do not change in the middle of a run), a log line prefix with the process id, and enough worker processes for parallel query.

`harness/docker/entrypoint.sh` (new)

Initializes the cluster on first start, appends the fuzzing settings, and runs the postmaster with all output (including `TRAP:` lines and assertion backtraces) appended to `/logs/postgresql.log`. Mount `/logs` from the host so the fuzzer's oracle can read it.

Finding during testing: the setting `backtrace_on_internal_error` does not exist in the current PostgreSQL `master` (the server refused to start). The scripts now check `postgres --describe-config` and only enable it when the server knows it.

harness/docker/coverage.sh (new)

Prints cumulative gcov coverage of the backend and PL/pgSQL as one JSON line: `{"lines_covered", "lines_total", "branches_covered", "branches_total", "functions_covered"}`. With `reset` it zeroes the counters and records the totals from an `--initial` capture, so the denominator stays fixed during a trial. Works with lcov 1.x and 2.x.

harness/docker/gdb_backtrace.sh (new)

Gets a symbolized backtrace of a crash **without core dumps and without ptrace permissions**: it copies the data directory, runs the SQL file in a single-user backend (`postgres --single`) under `gdb`, and prints the backtrace. A backtrace is the first thing maintainers ask for in a crash report.

harness/pgctl.py (new): Docker manager

```
python harness/pgctl.py build --ref master            # or --coverage, --sanitizer asan
python harness/pgctl.py up --workers 4                # containers pgfuzz-0..3 on ports 5432..54
python harness/pgctl.py coverage --worker 0 [--reset] # JSON line for the executor
python harness/pgctl.py backtrace --worker 0 repro.sql
python harness/pgctl.py info --worker 0               # version, commit, configure flags
python harness/pgctl.py down
```

harness/local/build_pg.sh and harness/local/pg_local.sh (new): no Docker, no root

`build_pg.sh` clones PostgreSQL at `PG_REF` and builds it with assertions (optionally coverage or a sanitizer) into `~/pgfuzz/install/<ref>`. PostgreSQL 17+ source needs bison and flex. If they are missing and there is no `sudo`, the script first builds m4, bison and flex into `~/pgfuzz/tools`.

`pg_local.sh` manages N local servers (`init`, `start`, `stop`, `status`, `coverage`, `backtrace`, `env`) on consecutive ports, each writing its log file next to its data directory.

Status on this machine: Docker is not installed, so the Docker image has not been built here yet. The local path works: in WSL Ubuntu 24.04 (20 cores, no sudo), the script bootstrapped m4/bison/flex and built **PostgreSQL master at commit `d733fca` with `--enable-cassert --enable-debug --with-libxml`** in about 13 minutes. The next step is to start the servers and connect from the Windows Python environment (psycopg 3.3.4 is installed).

harness/executor.py (fixed)

Member A's single-query tool now reads its settings from environment variables (`PGFUZZ_PSQL`, `PGHOST`, `PGPORT`, ...). It no longer contains hard-coded paths. It also reports a lost connection (psql exit code 2) as `CRASH`.

3.2 Evaluation framework (Member C's area)

New outcomes in the oracle (src/pgfuzz/core/types.py)

Two new bug-candidate outcomes, inserted into the severity order:

Outcome	Meaning
`wrong_result`	The same query returned a different set of rows under a different planner setting (differential oracle, see below).
`sanitizer`	AddressSanitizer or UndefinedBehaviorSanitizer reported a memory or undefined-behaviour error in the server log.

`StatementResult` gained two optional fields, `result_hash` and `variants`, which hold the facts the differential oracle needs. They are omitted from the logs when unused, so the log format stays compatible.

Differential plan testing (`src/pgfuzz/eval/differential.py`, new)

The idea: planner settings such as `enable_hashjoin`, `enable_nestloop`, `debug_parallel_query` or `work_mem` change *how* PostgreSQL runs a query, never *what* it returns. So the tool runs each eligible query once with default settings and once or twice under **plan variants**, and compares the results. A difference is a wrong-result bug in the optimizer or executor. This finds logic bugs that never crash.

- **17 plan variants:** nested loop only, hash join only, merge join only, no hash aggregation, no incremental/presorted sort, index scans only, sequential scans only, no memoize/material, forced parallel query, partition-wise join and aggregate, no partition pruning, GEQO, no join reordering, tiny `work_mem` (forces spill to disk), no GROUP BY/DISTINCT reordering, no self-join elimination, and JIT. Settings the server does not know are dropped automatically when the executor starts.
- **Few false alarms:** a query is compared only if its result is fully determined by the data. Excluded are `LIMIT`/`OFFSET`/`FETCH`, `TABLESAMPLE`, `DISTINCT ON`, row-locking clauses, volatile functions (`random()`, `clock_timestamp()`, `nextval()`, ...), order-sensitive aggregates without their own `ORDER BY` (`array_agg`, `string_agg`, `json_agg`, ...), and window functions that depend on the order among ties (`row_number`, `lag`, `first_value`, ...).
- Results are compared as **multisets** of PostgreSQL's own text output. **Float columns are masked** when the query aggregates, because the order of summation legitimately changes the last digits.
- The variant for a query is chosen **deterministically** from the query text, so a failure replays with the same variant. `materialize()` rewrites a failure into an explicit script (`SET ...;` then the query) for replay, minimization and bug reports.

Oracle (`src/pgfuzz/eval/oracle.py`)

- Detects AddressSanitizer, LeakSanitizer and UBSan reports in the server log.
- **Extracts C backtraces from the server log.** An assert-enabled PostgreSQL prints a backtrace after `TRAP:`. These frames are attached to the failure, so two crashes with the same top stack frames are counted as one bug ("stack" signature, the strongest kind) instead of many.
- Judges plan-variant facts: a variant that raised an internal error (`XX000`) becomes `internal_error`; a variant whose result digest differs becomes `wrong_result`; a lost connection during a variant is a crash. The variant name and its settings are stored with the failure.

Signatures (`src/pgfuzz/eval/signature.py`)

New keys for `wrong_result` (variant name plus the query's SQL features) and for sanitizer reports (tool, error kind, file). More generic frames are skipped (`errstart`, `BackendMain`, sanitizer internals, ...), so the five frames used for deduplication are the informative ones.

Executor (`src/pgfuzz/reference/pg_executor.py`)

- **Text-mode results:** every column is loaded as text, which fixes gap 9 above.
- `coverage_reset_command` runs before connecting at the start of every trial, which fixes gap 4.
- `differential: true` (or a mapping with `variants`, `per_query`, `max_rows`) runs the plan variants inside a savepoint that is rolled back, so no state leaks.
- Fixed the result-duplication bug (gap 10).
- Records the plan variants kept and dropped in the run manifest.

Runner (`src/pgfuzz/runner/trial.py`, `config.py`, `cli.py`)

- **Coverage time is no longer charged to the fuzzing budget** (gap 6). It is reported separately as `coverage_overhead_s`.
- New `{port}` placeholder (`base_port + worker`) for parallel workers, e.g. `dsn: "host=localhost port={port}"`.
- The trial summary now includes generator statistics (from an optional `stats()` method) and executor statistics (differential checks run).

Tests: the existing 58 unit tests still pass after these changes.

4. Detailed plan for the remaining work

4.1 Stage 2: grammar-aware generator (Member B's area)

The main technical contribution. Plan:

(a) A grammar language for SQL ("pgg" files) that people and an LLM can both read and edit. It looks like EBNF:

```
%types int bigint numeric float text bool date timestamptz interval jsonb int4range intarr texta
%start testcase

where_clause ::= "WHERE" expr_bool
expr_$T      ::= @col($T) <w=4> | @lit($T) <w=2> | "(" expr_$T ")" | case_$T | "(" scalar_sub_$T
case_$T      ::= "CASE" ("WHEN" expr_bool "THEN" expr_$T){1,3} ["ELSE" expr_$T] "END"
```

- terminals in quotes, rule names bare, `[...]` optional, `(...)` groups, `{m,n}`, `?`, `*`, `+` repetition, `<w=3>` alternative weights;
- **typed rules** through the `$T` template: `expr_$T` becomes `expr_int`, `expr_text`, ..., so generated expressions are type-correct;

- **semantic hooks** (`@table`, `@col(T)`, `@lit(T)`, `@items`, `@group_col`, `@new_table_name`, ...) implemented in Python. They know the schema and which tables are in scope;
- rule attributes `@scope` (new query block) and `@shape` (column count and types shared by `UNION` branches, `IN (subquery)`, `INSERT ... SELECT`).

(b) Two-phase generation. First build the *structure* (a derivation tree). Then *bind names* in a second pass over the finished tree, when every `FROM` item of the query block is known. This is how the `SELECT` list can refer to tables that appear later in the text, how correlated subqueries refer to outer queries, and how `JOIN ... ON` only uses tables to its left. Binding is deterministic per tree node, so mutating one subtree does not change the rest.

(c) The PostgreSQL grammar (`src/grammar/postgres.pgg`), covering the constructs where planner and executor bugs live: joins of all kinds, `LATERAL`, subqueries, CTEs (recursive, materialized, data-modifying), set operations, window functions with frames, aggregates (`FILTER`, `DISTINCT`, ordered-set), grouping sets, `DISTINCT ON`, JSON / SQL-JSON / `JSON_TABLE`, ranges, arrays, `MERGE` (including `RETURNING` and `NOT MATCHED BY SOURCE`), `INSERT ... ON CONFLICT`, `RETURNING old/new`, partitioned tables, generated columns (stored and virtual), indexes of all methods, views and materialized views, `PREPARE`/`EXECUTE` with forced generic plans, SQL and PL/pgSQL functions, triggers, rules, row-level security, extended statistics, and planner settings (`SET enable_*`, `work_mem`, GEQO, parallel query).

(d) Feedback. A corpus of "interesting" trees (trees that produced a new SQL feature, a new feature pair, a new runtime-error kind or a bug signature). New test cases are either fresh derivations or grammar-aware mutations of corpus trees: regenerate one subtree, or splice in a subtree of the same rule from another tree (as in Nautilus). Per-test-case code coverage is too expensive with gcov, so feature-pair novelty is the cheap proxy.

(e) A richer shared schema (`corpus/schema.sql` + `schema.yaml`): add tables with boolean, date, interval, array and varchar columns, hash and list partitioning, BRIN/GIN/hash indexes and extended statistics.

4.2 Stage 3: LLM-guided loop

- **LLM client** (`pgfuzz/llm/client.py`): any OpenAI-compatible API (OpenAI, OpenRouter, Groq, Together, DeepSeek, Gemini's OpenAI endpoint, a local Ollama) and the Anthropic API. The API key is read from an environment variable. Every response is cached on disk by prompt hash, so a trial can be replayed without network access, and an offline mode makes runs reproducible.
- **Grammar updater** (Member B, stage 3): the LLM receives a summary of the run so far (uncovered features, rules that cause the most semantic errors, bug signatures so far) and proposes edits in JSON: add a rule, add an alternative, change a weight. Each edit is parsed, checked and test-expanded before it is applied. Rejected edits and the reason are fed back in the next prompt.
- **LLM seed synthesis:** the LLM writes complete test cases over our schema, aimed at uncovered features and at **recently changed PostgreSQL code**. A helper (`pgfuzz focus`) turns `git log` of the PostgreSQL source into a hint file, so the LLM targets new features, where new bugs are most likely. The seeds are executed, kept when they are novel, and mutated further (literal edge values, join-type swaps, operator swaps, clause toggles).
- **Fair comparison:** LLM latency counts against the arm's budget. Calls, tokens and accepted edits are logged.

The three arms of the study stay exactly as proposed: **Random/Mutation → Grammar-only → Grammar + LLM**, plus optional ablations (grammar without feedback, LLM seeds only, LLM edits only).

4.3 Crash triage, minimization and reporting

- **`pgfuzz minimize`:** delta debugging (ddmin) over statements, then structure-aware reduction of the failing statement (drop clauses, list items, conjuncts, replace subqueries with `NULL`), then removal of the unneeded parts of the shared schema. The result is a *self-contained* script. The check is "same signature still reproduces", so the reduction never turns into a different bug.
- **`pgfuzz triage`:** group failures from all trials by signature, rank them (assertion > sanitizer > crash > internal error > wrong result), match them against the known-bug catalog, and write `triage.md` / `triage.csv`.
- **`pgfuzz bugreport`:** draft a pgsql-bugs report (version, commit, configure flags, minimized SQL, actual vs. expected behaviour, backtrace).
- **`harness/local/bisect.sh`:** automatic `git bisect` to find the commit that introduced the bug. A first-bad commit gets a report fixed much faster.

4.4 Configurations, tests, validation

- Configs: `stage2.yaml` (random vs. grammar), `stage3.yaml` (all three arms, coverage build), `hunt.yaml` (long bug-hunting campaign on assert-enabled `master` with the differential oracle and all generators).
- Unit tests for the grammar parser, generator, mutations, LLM loop (with a fake LLM), minimizer and differential oracle, plus an end-to-end smoke run on the simulated server.
- **Validation on the real server** built in WSL: run `selftest`, measure the grammar's parse-valid and valid-query rates, fix the rules that cause the most errors, then run a short fuzzing campaign and triage whatever it finds.

4.5 The three PDF guides you asked for

1. **PDF 1, Stage 1:** what Stage 1 is, every component, how to build and run it, testing, how failures are detected, deliverables.
2. **PDF 2, the whole project after Stage 2/3:** architecture, grammar language, generators, LLM loop, oracles, metrics, evaluation protocol, how to reproduce the headline result, what each team member owns.
3. **PDF 3, the bug-hunting playbook:** how to run long campaigns, which PostgreSQL versions and settings to target, how to use the grammar and the LLM to aim at new features, how to triage, minimize, bisect and verify a candidate, how to write a pgsql-bugs report that maintainers accept, how to follow up, and how to record the result in the README and the paper.

5. Honest status summary

Item	Status
Stage 1 gaps fixed (harness, oracle, executor, runner)	Done (code written, unit tests pass)
Docker image	Written, **not built yet** (no Docker on this machine)
Local assert-enabled PostgreSQL master	**Built** (commit `d733fca`); servers being started
Differential wrong-result oracle	Written, not yet run against the real server
Grammar language, engine, PostgreSQL grammar	Next
LLM loop, minimizer, triage, bug reports	After the grammar
Experiments, measured numbers	Not run yet; numbers will come only from real runs
Bugs found	None yet

What the team should do in parallel now: read `src/README.md` and this report; check that Docker builds on a Linux machine (`python harness/pgctl.py build --ref master`); and start reading recent pgsql-bugs threads to see what accepted fuzzer reports look like.